

Short Answer type

- 1) $E \rightarrow E + E / id$ it is Ambiguous because $E \rightarrow E + E$ here two non-terminals are so it creates the problem. $\alpha B \beta$

3) $id + id \neq id$

$$t_1 = id * id$$

$$t_2 = id$$

$$id + id * id = t_2 + t_1$$

- 4) left recursion from $A \rightarrow A \alpha | \beta$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A'$$

5) $S \rightarrow iETS / iEtSes / a$

By Applying left factoring we need to remove the Ambiguity

$iETS$ is same

$$S \rightarrow \underline{iETS} / \underline{iEtSes} / a$$

$$S \rightarrow es'$$

$$S \rightarrow a$$

52
100

6) $S \rightarrow aBDb$ calculate first and follow

$B \rightarrow cC$
 $C \rightarrow bc/E$
 $D \rightarrow EF$
 $E \rightarrow g/E$
 $F \rightarrow f/E$

$\text{First}(S) \rightarrow \{a, \phi\}$

$\text{First}(B) \rightarrow \{c\}$

$\text{First}(C) \rightarrow \{b\}$

$\text{First}(D) \rightarrow \{\phi\}$

$\text{First}(E) \rightarrow \{g, \epsilon\}$

$\text{First}(F) \rightarrow \{f, \epsilon\}$

Follow

$\text{Follow}(S) = \{\phi\}$

$\text{Follow}(B) = \{g, f, \phi\}$

$\text{Follow}(C) = \{g, f, \phi\}$

$\text{Follow}(D) = \{g, f, \phi\}$

$\text{Follow}(E) = \{g, f, \phi\}$

$\text{Follow}(F) = \{g, f, \phi\}$

7)

$S \rightarrow Sa|sb|cd$

$S \rightarrow cd$

Here S is common, $S \rightarrow \underline{Sa}/\underline{sb}/cd$

$S \rightarrow as'$

$S \rightarrow bs'$

So, the grammar contains left recursion
Intermediate

After solving we will get $S \rightarrow as'$

$S \rightarrow bs'$

$S \rightarrow cd$

Given; $E \rightarrow TE'$

$E' \rightarrow +TE' / E$

The ~~First~~ (E') = ?

$\text{First}(E') = \{+, \epsilon\}$

The string (1) is valid because $(\rightarrow \epsilon$ is a terminal

SHIFT REDUCING PARSING

- ① We will be considering the priority order.
- ② The priority order follows $\text{id}, (,), +, -, E, \epsilon, \$$
Decreasing in priority
- ③ If the token is less than the left side production then you need to reduce
- ④ If it is greater you need to shift.
- ⑤ At last in the end you need to get the \$
- ⑥ This is how shift Reducing parsing happens.

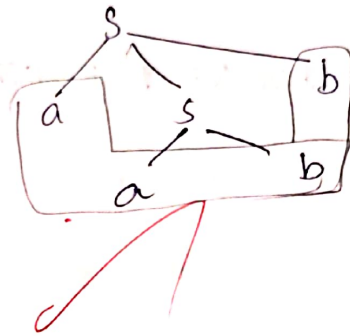
2

Q2) $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$, for grammar $S \rightarrow asb/ab$
 $aabb$ is accepted or not?

(i) Set for non-terminals are $\{A, B, S\}$

(ii) $S \rightarrow asb$
 $S \rightarrow ab$

$aabb$ is not accepted



First(S) :- $\{a, b\}$
 First(A) :- $\{a\}$
 First(B) :- $\{b\}$

(3)

$S \rightarrow A$
 $A \rightarrow aB/Ad$
 $B \rightarrow b$
 $C \rightarrow g$

First(S) :- $\{a\}$
 First(A) :- $\{a\}$
 First(B) :- $\{b\}$
 First(C) :- $\{g\}$

Follow(S) :- $\{\$ \}$
 Follow(A) :- $\{\$ \}$
 Follow(B) :- $\{\$ \}$
 Follow(C) :- $\{\$ \}$

$S \rightarrow A$
 $A \rightarrow aBA'$
 $A' \rightarrow dA'$
 $B \rightarrow b$
 $C \rightarrow g$

$\text{first}(S) := \{a\}$
 $\text{first}(A) := \{a\}$
 $\text{first}(A') := \{d\}$
 $\text{first}(B) := \{b\}$
 $\text{first}(C) := \{g\}$

$\text{follow}(S) := \{\$ \}$
 $\text{follow}(A) := \{\$ \}$
 $\text{follow}(A') := \{\$ \}$
 $\text{follow}(B) := \{d, \$ \}$
 $\text{follow}(C) := \{d, \$ \}$

16)

$S \rightarrow AB$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid c$

↓

$B \rightarrow bBB'$

$B' \rightarrow CB'$

The Non-terminals are $\{A, B\}$

$\text{follow}(A) = \{\$ \}$

$\text{follow}(B) = \{c, \$ \}$

17

Left Recursion

The rule for left Recursion is simple that if you find the same Non-terminals in production you need to eliminate it.

$A \rightarrow A\alpha \mid \beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Left Factoring

First identify if there are same terms in the given question and do consider it 1 Eg:

$S \rightarrow \underline{iEtS} / \underline{iEtSes} / a$

$S \rightarrow es'$

$S \rightarrow a$

— this is how left factoring works.

18) $id + id * id$

Grammar

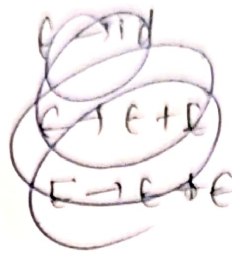
$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *ET' / \epsilon$$

$$F \rightarrow (E) / id$$



first(E) :- { C, id }

first(E') :- { + }

first(T) :- { C, id }

first(T') :- { * }

first(F) :- { C }

Follow(E) :- { \$ }

follow(E') :- { \$ }

follow(T) :- { +, \epsilon, \$ }

follow(T') :- { +, \epsilon, \$ }

follow(F) :- { (, +, \epsilon, \$ }

Nonterminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$				$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'			$T' \rightarrow *ET'$			$T' \rightarrow \epsilon$
F				$F \rightarrow (E) / id$		

Shift reduce Parsing

Stack

PLP

$\epsilon \rightarrow id$
 $\epsilon \rightarrow E + E$
 $\epsilon \rightarrow E * E$

\$

id + id + id \$

shift

\$ id

+ id + id \$

Reduce $\epsilon \rightarrow id$

\$ E

+ id + id \$

shift

\$ E +

id * id \$

shift

\$ E + id

* id \$

Reduce $\epsilon \rightarrow id$

\$ E + E

+ id \$

Reduce $\epsilon \rightarrow E + E$

\$ E

* id \$

shift

\$ E *

id \$

shift

\$ E * id

\$

Reduce $\epsilon \rightarrow id$

\$ E * E

\$

Reduce $\epsilon \rightarrow E * E$

\$ E

\$

—

20)

```
#include <stdio.h>
```

```
int main() {
```

```
    int a=10, b=20;
```

```
    int sum = a+b;
```

```
    printf("%d", sum);
```

```
    return 0;
```

```
}
```

a) The phases of Compiler are:

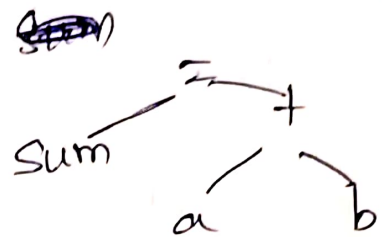
1) Lexical Analyser

The tokens are $a, b, \text{sum}, \{, \}, (,), ;$

2) Syntax Analyser

which is also called parse tree

$\text{Sum} = a + b$



3) Semantic Analyser

$t_1 = a + b$
 $\text{sum} = t_2$
 $t_2 = t_1$

4) Intermediate Code Generator

It gives lexical code.

5) Code Analyser

It analyses the code whether it is true or false.

6) Code optimizer

It optimizes the code.

b) $\text{Sum} = a + b$

